Career Switch Prediction: A Machine Learning Approach to Identify Potential Employee Turnover

Abstractâ€” Employee turnover and career switching pose significant challenges and costs to organizations worldwide. This project presents a machine learning approach to predict whether an individual is likely to change their career based on demographic, educational, and professional data. We rigorously process an imbalanced dataset of 5,000 records, addressing missing values, categorical encoding, and feature scaling without data leakage. Six supervised learning algorithms (K-Nearest Neighbors, Decision Tree, Logistic Regression, Linear Regression, Naive Bayes, Neural Network) and one unsupervised algorithm (K-Means) are trained, evaluated, and compared. The results indicate that the Neural Network and Logistic Regression yield the strongest predictive performance, achieving ROC-AUC scores of ~0.75.

Keywordsâ€” Machine Learning, Career Switch Prediction, Classification, Pre-processing, Neural Networks.

I. Introduction

The modern job market is highly dynamic, with professionals frequently transitioning between different career paths. For human resource (HR) departments, anticipating whether a candidate or employee is looking to switch careers is critical for tailoring recruitment, training, and retention strategies.

The objective of this project is to build robust predictive models capable of determining whether an individual will change their career. We treat this as a binary classification problem, where the target variable `will_change_career` denotes whether a switch will occur (1) or not (0). The motivation behind this project is to automate and optimize HR screening processes by uncovering hidden patterns in candidate profiles.

II. Dataset Description

The analysis is based on the `Career_Switch_Prediction_Dataset.csv`. A thorough Exploratory Data Analysis (EDA) was performed to understand the structure and properties of the data.

- **Data Points and Features:** The dataset consists of 5,000 instances and 14 features (13 predictors and 1 target).
- **Problem Type:** This is distinctly a classification problem because the objective is to predict a discrete binary label (`will_change_career`).
- **Feature Types:** The dataset contains a mix of quantitative (3 features, e.g., training hours) and categorical features (11 features, e.g., gender, education level, major discipline).
- **Class Imbalance:** The target variable is significantly imbalanced. There are 3,738 instances belonging to Class 0 (No Career Switch) and 1,262 instances belonging to Class 1 (Will Switch Career). A bar chart generated in the workflow confirmed an approximate 3:1 ratio favoring the negative class.
- **Correlation:** A correlation heatmap generated via the `seaborn` library revealed the internal relationships among numerical variables. It was observed that `enrollee_id` possessed negligible correlation with the target and functioned merely as a unique identifier; hence, it was dropped.

III. Dataset Pre-processing

To ensure model validity and prevent data leakage, all imputation and scaling parameters were learned exclusively from the training data after the dataset was split.

- **Problem 1: Null / Missing Values**
- *Context:* A high volume of missing values was detected in critical categorical features such as `gender`, `major_discipline`, and `company_size`.
- *Solution:* We applied targeted imputation. For numerical columns, missing values were replaced with the Median of the respective column in the training set. For categorical columns, we utilized the Mode (most frequent value) of the training set.
- **Problem 2: Categorical Values**

- *Context:* Standard machine learning algorithms require strictly numerical input, yet the majority of our predictors were categorical strings.
- *Solution:* We applied `LabelEncoder` to systematically map string labels into integer representations.
- **Problem 3: Feature Scaling**
- *Context:* Variance analysis showed that features like `training_hours` had drastically different scales compared to encoded categorical features. This discrepancy heavily biases distance-based algorithms like KNN and slows convergence in Neural Networks.
- *Solution:* We applied standard scaling (`StandardScaler`) to transform the features such that they exhibit a mean of 0 and a standard deviation of 1.

IV. Dataset Splitting

To evaluate the models authentically, the dataset was partitioned into an 80% Training set and a 20% Testing set using stratified sampling to preserve the target class ratios.

To combat the aforementioned class imbalance, random undersampling was applied *strictly to the training set*. The majority class was reduced to match the minority class size. The test set was left untouched to ensure that the final evaluations reflect real-world, imbalanced conditions.

V. Model Training & Testing (Supervised and Unsupervised)

Both unsupervised and supervised algorithms were implemented using the `scikit-learn` framework.

1. **Unsupervised Learning:** We treated the problem as an unsupervised learning task by applying the **K-Means** clustering algorithm (K=2) to the training features. To showcase the clusters visually, Principal Component Analysis (PCA) was used to project the multi-dimensional data into a 2D scatter plot.
2. **Supervised Learning:** The following six models were trained on the balanced training set and tested on the unseen test set:
3. K-Nearest Neighbors (KNN)
4. Decision Tree
5. Logistic Regression
6. Linear Regression
7. Naive Bayes
8. Neural Network (Multi-Layer Perceptron)

VI. Model Selection & Comparison Analysis

The six models were compared utilizing the test set across multiple rigorous metrics. Linear Regression, inherently a continuous predictor, was evaluated utilizing regression metrics, and subsequently thresholded at 0.5 to allow comparison via classification metrics.

A. Classification Metrics Comparison

- **KNN:** Accuracy: 63.80% | Precision: 38.07% | Recall: 61.27% | AUC: 0.6729
- **Decision Tree:** Accuracy: 63.90% | Precision: 36.06% | Recall: 51.78% | AUC: 0.5985
- **Logistic Regression:** Accuracy: 70.90% | Precision: 44.42% | Recall: 64.82% | AUC: 0.7516
- **Linear Regression (Thresholded):** Accuracy: 71.80% | Precision: 45.71% | Recall: 67.59% | AUC: 0.7523
- **Naive Bayes:** Accuracy: 69.70% | Precision: 42.86% | Recall: 63.24% | AUC: 0.7303
- **Neural Network:** Accuracy: 72.90% | Precision: 46.82% | Recall: 63.64% | AUC: 0.7497

B. Regression Metrics for Linear Regression

- **RÂ² Score:** 0.1601
- **Loss (MSE):** 0.1610

C. Visual Comparisons

Within the codebase, several visual comparisons were generated: * A **Bar Chart** successfully showcased the prediction accuracy across all six models. * **Confusion Matrices** were mapped using heatmaps, detailing True Positives, True Negatives, False Positives, and False Negatives for each architecture. * **ROC Curves** mapped the True Positive Rate against the False Positive Rate, visually confirming that the Neural Network and Logistic Regression possessed the greatest Area Under the Curve (AUC).

VII. Conclusion

- **Understanding from Results:** The comparative analysis highlights that generalized linear models (Logistic Regression) and deep learning architectures (Neural Networks) are the most capable of deciphering the complex feature interactions driving career transitions. The Neural Network achieved the peak accuracy (72.90%), while Logistic Regression maintained an excellent balance of Recall and AUC.
- **Performance Comments:** The decision to randomly undersample the training data proved highly effective. It forced the models to prioritize the minority class, resulting in meaningful Recall scores (up to 67%). While this naturally depresses absolute accuracy compared to a naive model that only guesses the majority class, it is fundamentally more useful for HR practitioners who specifically need to identify career switchers.
- **Reasons for Results:** The models' stability is directly attributable to rigorous preprocessing. Crucially, strictly preventing data leakage by fitting the imputers and scalers exclusively on the training data ensured the test metrics were highly authentic. Linear Regression performed suboptimally as a fundamental classifier due to

its attempt to fit a hyperplane minimizing MSE, rather than utilizing a log-odds threshold optimized for binary outcomes.

- **Challenges Faced:** The primary difficulty encountered was managing a substantial volume of missing categorical data while preventing data leakage across the train-test boundary. Employing the statistical mode for imputation was a pragmatic solution, though it inherently introduces a minor bias toward the dominant categories. Evaluating Linear Regression on a classification task also required custom thresholding to ensure parity in metric comparison.

References

[1] Scikit-learn developers, "Scikit-learn: Machine Learning in Python," *scikit-learn.org*. [2] "CSE422 Lab Learning Materials," course documents and Jupyter notebooks.